Zihao (Jason) Zhang

Merced, CA | jasonzhang5@ucmerced.edu | linkedin.com/in/jasonzzh | tacorav4.github.io

Professional Summary

M.S. candidate in Cognitive and Information Sciences with national laboratory experience training and evaluating neural network models inside an established scientific codebase. Builds reproducible ML pipelines at scale in Python and Bash/Slurm and debugs scientific Java at the source level, with emphasis on leakage-safe experimental design and honest model evaluation.

Education

University of California, Merced — Merced, CA
Cognitive and Information Sciences, M.S. | GPA: 4.0/4.0 — *Expected Aug 2026*

University of California, Davis — Davis, CA
Cognitive Science, B.S. — *Jun 2025*

Experience

Affiliate Intern, Machine Learning and Metagenomics — May 2026 – Jul 2026
Joint Genome Institute, Lawrence Berkeley National Laboratory — *Berkeley, CA*

- Built the QuickBin PacBio neural-network adaptation workflow, creating reproducible Bash/Slurm pipelines around BBTools/QuickBin, Flye, minimap2, and GradeBins on Dori HPC with FASTA/SAM/TSV validation at each stage.
- Designed and scaled synthetic long-read benchmarks from 663 to 2,013 genomes (8.29 Gbp, 62,714 contigs, N50 413,899 bp); the 663-genome pack included 50 related strains at 90–95% ANI to stress-test separation of similar organisms.
- Enforced genome-level train/validation/test splits (1,435/279/299) before generating any training vectors, preventing the pair-level leakage a post-hoc split silently introduces. An automated cross-split leakage check confirmed zero leaked pairs.
- Retrained the shipping classifier through hyperparameter sweeps. On the genome-held-out evaluation split used in candidate retention, AM1 cut contamination from 1.7530 to 1.3724 for a 0.857-point recovery cost.
- Ran the project's first external evaluation on CAMI II PacBio sample 14, where model ordering reversed across a changed dataset, truth universe, and runtime. Reported the confounded result rather than claiming a simple generalization.
- Directed an AI-assisted Java source investigation of BBTools/QuickBin, defining validation contracts for an isolated patch that restored training-vector output. Separately audited a FASTA/SAM and CIGAR repair that recovered 74,895 alignments in a 10-genome pilot.

Research Assistant — Jun 2024 – Jun 2025
Janata Lab — *Davis, CA*

- Developed a real-time eye-blink detection module integrating browser-based HTML/JavaScript workflows with Python detection logic for behavioral data collection.
- Built experiment data pipelines with Django ORM and PyEnsemble to manage datasets and enforce data-integrity constraints.

Projects

Tonal Inference Modeling — Aug 2025 – Present
Python, PyTorch

- Compared an Elman SRN against an EMA+MLP baseline for MIDI key inference. On two real pieces, changing the representation/decoder path shifted behavior more than replacing EMA with SRN inside the chord-id path.
- Created a six-piece, six-level MIDI benchmark. Ran a predeclared 54-condition timestep-gate sensitivity audit against it with frozen model variants and L1/L6 stability controls, mapping the gate family’s damage-recovery frontier.

AI Algorithms and Reinforcement Learning Projects — Jan 2025 – Aug 2026
Python | 2025 coursework, independently re-verified 2026

- Implemented A* with elevation-aware admissible heuristics and a Dijkstra baseline. On synthetic terrain, optimal A* matched Dijkstra's path cost while expanding 81.25% fewer nodes. Weighted A* (w = 1.8) cut expansions 98.57% for a 1.83% cost increase.
- Recovered a Minimax + Alpha-Beta Connect Four agent and traced two prior benchmark results to measurement artifacts rather than algorithm behavior. With terminal detection added, it scored 93/100 on held-out seeds against an unmodified 1001-rollout Monte Carlo baseline at fixed depth 6.
- Implemented a DQN for Pong with replay buffering, target-network updates, and epsilon-greedy exploration.

Skills

ML and Applied AI: PyTorch, scikit-learn, Pandas, NumPy, Neural Network Training, Hyperparameter Search, Model Evaluation, Benchmark Design
Programming: Python, Java, C/C++, SQL, R, Bash
Systems and Workflow: Linux, Git, Slurm, HPC, Django ORM, Reproducible Pipelines, Integration Testing, Object-Oriented Design